

Trei abordari metaeuristice hibride pentru problema colorarii grafurilor

Lefter Sorin | Bolohan Constantin-Sebastian

FII, UAIC Iasi

Lucrarea compara trei paradigme metaeuristice hibride pentru problema fixed- k
graph coloring: cautare evolutionara, inteligenta de roi si cautare locala
adaptiva cu mecanisme de breakout.

28 mai 2026

Cuprins

1	Introducere	1
2	Descrierea problemei	1
3	Benchmarks	2
3.1	Instante usoare	3
3.2	Instante medii	3
3.3	Instante dificile	3
4	Metoda 1: Algoritm Genetic Hibrid de tip memetic	4
4.1	Descrierea metodei	4
4.2	Pseudocod	6
4.3	Parametrii folositi si de ce conteaza	6
4.4	Rezultatele metodei 1	8
5	Metoda 2: Algoritm Hibrid bazat pe Ant Colony Optimization	9
5.1	Descrierea metodei	9
5.2	Pseudocod	10
5.3	Parametrii folositi si de ce conteaza	10
5.4	Rezultatele metodei 2	12
6	Metoda 3: Cautare Locala Hibrida cu Breakout Tabu	13
6.1	Descrierea metodei	13
6.2	Pseudocod	15
6.3	Parametrii folositi si de ce conteaza	15
6.4	Rezultatele metodei 3	17
7	Comparatie intre cele trei metode	17
8	Concluzie	18
9	Bibliografie	20

1 Introducere

Problema colorarii grafurilor reprezinta una dintre cele mai cunoscute probleme de optimizare combinatoriala dificila, cu aplicatii in planificare, alocarea resurselor, frecvente radio sau alocarea registrelor.

In formularea clasica, obiectivul este determinarea unui numar minim de culori necesare pentru a colora un graf astfel incat doua varfuri adiacente sa nu primeasca aceeasi culoare. Aceasta valoare este cunoscuta drept numarul cromatic al grafului si este dificil de determinat exact pentru instante de dimensiuni medii sau mari.

In aceasta lucrare se abordeaza o strategie iterativa de cautare a unei valori cat mai mici pentru k . Mai precis, algoritmi pornesc de la o limita superioara initiala obtinuta printr-o euristica de tip DSATUR si incearca succesiv sa gaseasca colorari fezabile pentru valori descrescatoare ale lui k .

Pentru fiecare valoare a lui k , problema este tratata ca o problema de optimizare a conflictelor, in care se minimizeaza numarul de muchii conflictuale. Daca se obtine o solutie fara conflicte, valoarea lui k este redusa, iar procesul continua. In caz contrar, algoritmul se opreste, considerand ca limita inferioara a fost atinsa sau ca resursele disponibile nu permit imbunatatiri suplimentare.

Aceasta abordare permite combinarea eficientei metaeuristicilor pentru problema fixed- k cu un mecanism global de cautare a numarului minim de culori.

In cadrul lucrarii sunt analizate trei metode hibride:

- un algoritm genetic memetic, care opereaza direct asupra colorarilor si integreaza mecanisme de recombinare structurala si cautare locala;
- un algoritm hibrid bazat pe *Ant Colony Optimization*, care construiește ordonari de varfuri utilizate ulterior in procesul de colorare;
- o metoda hibrida de cautare locala adaptiva, bazata pe Tabu Search, penalizari de tip breakout, restarturi strategice si memorie de elite.

Scopul lucrarii este atat evaluarea performantelor acestor metode, cat si justificarea alegerilor de proiectare, cu accent pe rolul parametrilor si al mecanismelor de hibridizare in obtinerea unor solutii de calitate.

2 Descrierea problemei

Fie un graf neorientat $G = (V, E)$, unde V reprezinta multimea varfurilor, iar E multimea muchiilor. Pentru o valoare fixata k , dorim sa construim o functie de colorare

$$C : V \rightarrow \{0, 1, \dots, k - 1\}$$

astfel incat pentru orice muchie $(u, v) \in E$ sa fie indeplinita conditia $C(u) \neq C(v)$.

Daca pentru valoarea curenta a lui k nu se poate obtine o colorare complet fezabila, atunci

obiectivul devine minimizarea numarului de conflicte. In acest caz, functia obiectiv este:

$$f(C) = \sum_{(u,v) \in E} \mathbb{I}[C(u) = C(v)],$$

unde $\mathbb{I}[\cdot]$ este functia indicator. O solutie este fezabila daca si numai daca $f(C) = 0$.

Aceasta formulare este utila deoarece permite tratarea problemei ca pe un proces gradual de optimizare. Chiar daca fezabilitatea nu este obtinuta imediat, calitatea unei solutii poate fi evaluata prin numarul de muchii conflictuale ramase.

Reprezentarea solutiei si a grafului

Din punct de vedere al implementarii, fiecare solutie este reprezentata printr-un vector de dimensiune $|V|$, in care pozitia i memoreaza culoarea atribuita varfului v_i . Aceasta reprezentare este directa, simpla si foarte potrivita pentru operatori precum recolorarea, mutatia sau schimbarea locala a unei solutii.

Graful este stocat prin liste de adiacenta. Alegerea nu este intamplatoare, ci este motivata de eficienta calculului local. Atunci cand culoarea unui singur varf se modifica, doar muchiile incidente acelui varf isi pot schimba starea de conflict. Prin urmare, variatia functiei obiectiv poate fi calculata in timp $O(\deg(v))$, in loc de $O(|E|)$.

De ce conteaza evaluarea incrementala

Un element central al lucrarii este distinctia dintre:

- **evaluare completa**, in care se recalculeaza numarul total de conflicte din intreaga solutie;
- **evaluare incrementala**, in care se actualizeaza doar efectul unei modificari locale.

Aceasta diferenta este esentiala deoarece bugetul de evaluari complete este strict limitat. In consecinta, algoritmi trebuie sa foloseasca evaluarile globale doar in momentele importante, cum ar fi:

- initializarea unei solutii noi;
- evaluarea unui descendent dupa operatori structurali;
- validarea unor schimbari majore.

In schimb, mutatiile locale, procedurile de reparare si cautarea tabu se bazeaza predominant pe actualizari delta, mult mai ieftine. Aceasta alegere permite intensificarea cautarii fara a depasi constrangerea impusa de bugetul de evaluari.

3 Benchmarks

Pentru evaluarea celor trei metode a fost utilizat un set comun de grafuri benchmark consacrate in literatura pentru *Graph Coloring Problem*. Selectia include grafuri cu structuri

diferite: instante sintetice, grafuri bazate pe constrangeri, grafuri dense aleatoare si exemple clasice din suita DIMACS/COLOR.

Aceasta varietate este importanta deoarece dificultatea problemei nu depinde doar de numarul de varfuri, ci si de densitate, regularitate, numarul cromatic si forma peisajului de cautare. Din acest motiv, instantele au fost grupate dupa dificultatea empirica observata in literatura si confirmata prin testele preliminare.

3.1 Instante usoare

Categoria usoara contine grafuri relativ mici sau puternic structurate, pentru care euristicele constructive obtin de regula rezultate foarte bune. Exemple reprezentative sunt `queen7_7`, derivat din problema damelor, si `myciel3`, un graf Mycielski mic, cu o structura teoretic interesanta, dar inca usor de tratat euristic.

3.2 Instante medii

Instantele medii sunt suficient de complexe pentru a evidentia diferenta dintre o metoda simpla si una robusta. In aceasta categorie intra `myciel4`, `flat300_28_0`, `1e450_5a`, `fpsol2.i.1` si `queen11_11`. Aceste grafuri permit observarea modului in care algoritmi gestioneaza simultan marimea, structura si constrangerile cromatice.

3.3 Instante dificile

Categoria dificila include grafuri mari sau foarte dense, unde fezabilitatea devine greu de atins la valori apropiate de pragul cromatic. Grafurile din familia DSJC, in special `DSJC500.9` si `DSJC1000.9`, sunt exemple clasice de instante ostile pentru metaeuristici. De asemenea, `inithx.i.1` este cunoscuta pentru optime locale inselatoare, iar `DSJC125.9`, desi mai mica, este extrem de densa si dificil de colorat la limita.

Tabela 1: Caracteristicile instantelor benchmark, ordonate dupa dificultate

Instanta	Varfuri	Structura	k_{ref}	Dificultate
queen7_7	49	bazata pe constrangeri	7**	Usoara
myciel3	11	constructie Mycielski	4**	Usoara
huck	74	Stanford GraphBase	11**	Usoara
myciel4	23	constructie Mycielski	5**	Medie
flat300_28_0	300	graf structurat de tip flat	28**	Medie
le450_5a	450	alocare de registre	5**	Medie
fpsol2.i.1	496	derivata din aplicatii	65**	Medie
queen11_11	121	bazata pe constrangeri	11**	Medie
inithx.i.1	864	bazata pe constrangeri	54**	Dificila
DSJC500.5	500	aleatoare, densitate 0.5	47*	Dificila
DSJC125.9	125	aleatoare, densitate 0.9	44*	Dificila
DSJC500.9	500	aleatoare, densitate 0.9	126*	Dificila
DSJC1000.9	1000	aleatoare, densitate 0.9	223*	Foarte dificila

** numar cromatic exact raportat in benchmark-urile DIMACS/COLOR.

* cea mai buna limita superioara cunoscuta in literatura, cand valoarea exacta nu este cunoscuta.

Toate instantele au fost testate in acelasi regim experimental si sub acelasi buget de evaluari pentru cele trei metode, astfel incat comparatia sa fie corecta si relevanta.

4 Metoda 1: Algoritm Genetic Hibrid de tip memetic

4.1 Descrierea metodei

Prima metoda propusa este un algoritm genetic memetic proiectat special pentru varianta *fixed-k* a problemei colorarii grafurilor, in conditiile unui buget strict de evaluari. Ideea centrala este combinarea a doua componente complementare:

- **cautarea globala**, realizata prin mecanisme populationale specifice algoritmilor genetici;
- **intensificarea locala**, realizata prin operatori de reparare si cautare tabu.

Fiecare individ din populatie reprezinta o colorare completa a grafului. Populatia initiala este generata printr-o euristica de tip DSATUR, astfel incat algoritmul sa porneasca din solutii mai bune decat unele obtinute complet aleator. Pentru a nu sacrifica diversitatea, aceste solutii sunt apoi perturbate controlat.

Selectia parintilor se face prin turneu, iar recombinarea nu este una standard, aplicata independent pe pozitii, ci una orientata structural. Mai precis, este folosit un crossover bazat pe taieturi aleatoare ale grafului. Descendentul mosteneste culoarea varfurilor de la

un parinte pe o parte a taieturii si de la celalalt parinte pe partea complementara. Aceasta idee permite transferul unor substructuri coerente, nu doar al unor culori izolate.

Dupa recombinare pot aparea doua tipuri de probleme:

- conflicte noi pe muchiile care traverseaza taietura;
- nealiniera etichetelor de culoare intre cei doi parinti.

Pentru a reduce al doilea efect, metoda aplica periodic o armonizare a etichetelor prin algoritmul ungar. In acest fel, doua clase de culoare care sunt structural similare, dar codificate diferit in cele doua solutii-parinte, pot fi realiniate inainte ca descendentul sa fie evaluat.

Mutatia este orientata catre varfurile aflate in conflict. In locul unei mutatii uniforme si oarbe, sunt aplicate recolorari ghidate de conflicte, iar intensitatea acestora se adapteaza in functie de progresul sau stagnarea cautarii. In apropierea fezabilitatii, metoda devine mai agresiva pe partea de intensificare pentru a elimina conflictele reziduale.

Componenta de cautare locala este un *Iterated Greedy Local Search* asistat de tabu. Acesta nu este folosit ca algoritm independent, ci ca operator de reparare aplicat selectiv solutiilor promitatoare. Avantajul principal este ca poate accepta controlat si miscari neimbunatatitoare, ceea ce ii permite sa iasa din blocaje locale.

In ansamblu, metoda 1 se bazeaza pe urmatoarea idee: pentru instante dificile, performanta buna apare atunci cand recombinarea structurala, mutatia adaptiva si repararea locala sunt integrate intr-un mod coerent si atent controlat de bugetul disponibil.

4.2 Pseudocod

Algorithm 1 Algoritm genetic memetic pentru colorarea fixed- k

Require: Graful $G = (V, E)$, numarul de culori k , buget de evaluari B

Ensure: Cea mai buna colorare gasita

```

1: Initializeaza populatia folosind constructie DSATUR
2: Evalueaza indivizii initiali si memoreaza cea mai buna solutie
3: while conditiile de oprire nu sunt indeplinite do
4:   Selecteaza perechi de parinti prin selectie de tip turneu
5:   Genereaza taieturi aleatoare pentru crossover
6:   Construiesc descendentii prin crossover care pastreaza structura
7:   Aplica mutatie adaptiva orientata pe conflicte
8:   if generatia curenta este multiplu de  $N_h$  then
9:     Aplica armonizarea etichetelor de culoare cu algoritmul ungar
10:  end if
11:  Evalueaza descendentii
12:  if intensificarea locala este activata then
13:    Aplica reparare locala IGLS asistata de tabu
14:  end if
15:  Inlocuieste indivizii slabi cu descendentii promitatori
16:  Adapteaza intensitatea mutatiei in functie de progres sau stagnare
17:  if se detecteaza stagnare prelungita then
18:    Injecteaza diversitate sau executa restart partial al populatiei
19:  end if
20: end while
21: return cea mai buna solutie gasita

```

4.3 Parametrii folositi si de ce conteaza

Comportamentul algoritmului genetic este controlat de un set restrans de parametri cu impact major. Selectia lor nu este arbitrara. Fiecare valoare a fost aleasa astfel incat sa mentina un echilibru intre patru obiective:

- pastrarea diversitatii in populatie;
- intensificarea eficienta in zone promitatoare;
- limitarea costului computational;
- cresterea robustetii pe instante dificile.
- **Dimensiunea populatiei ($N = 180$, cu restrictia $N \leq 200$).** O populatie mai mare inseamna diversitate mai buna si risc mai mic de convergenta prematura. Totusi, fiecare generatie devine mai costisitoare. Valoarea aleasa mentine varietatea solutiilor fara sa consume excesiv bugetul care trebuie rezervat si pentru operatori mai scumpi.

- **Numarul de elite ($E = 2$).** Elitismul previne pierderea celor mai bune solutii. O valoare prea mare ar reduce diversitatea, iar una prea mica ar face progresul fragil. Alegerea $E = 2$ este suficienta pentru stabilitate, fara a rigidiza populatia.
- **Numarul maxim de generatii ($G_{max} = 2000$).** Aceasta limita functioneaza in principal ca prag de siguranta. In practica, oprirea este determinata mai ales de bugetul de evaluari sau de stagnare.
- **Bugetul de evaluari complete ($B = 400000$).** Acest parametru este central pentru intreaga lucrare. Evaluările complete sunt scumpe, mai ales pe grafuri dense, motiv pentru care metoda se bazeaza puternic pe actualizari incrementale. Bugetul forteaza utilizarea disciplinata a evaluarilor globale.
- **Dimensiunea turneului ($t = 3$).** Un turneu prea mare creste excesiv presiunea de selectie si favorizeaza convergenta prematura. Un turneu prea mic incetinesc progresul. Valoarea 3 ofera un compromis potrivit.
- **Numarul de taieturi candidate per generatie (32).** Acestea asigura diversitate in recombinare fara a introduce un overhead prea mare. Este suficient pentru a produce descendenti diferiti, dar nu atat de mare incat sa devina costisitor.
- **Frecventa armonizarii ungare ($N_h = 3$ generatii).** Armonizarea etichetelor corecteaza nealinierea culorilor dupa crossover. Deoarece algoritmul ungar este mai scump, aplicarea sa la fiecare trei generatii reprezinta un compromis bun intre calitatea recombinarii si cost.
- **Bugetul mutatiei pe individ ($M_{base} = \max(2, |V|/200)$, cu plafon $\max(5, |V|/100)$).** Mutatia trebuie sa fie proportionala cu dimensiunea instantei. Pe grafuri mici sunt suficiente putine miscari, iar pe grafuri mari o mutatie prea slaba devine irelevantă.
- **Intervalul pentru intensitatea adaptiva a mutatiei ($I(t) \in [0.5, 3.5]$).** Cand cautarea progreseaza, mutatia trebuie temperata pentru a rafina solutiile. In stagnare, ea trebuie amplificata pentru a forta explorarea. Limitele impiedica atat colapsul mutatiei, cat si distrugerea excesiva a structurii.
- **Pragul de endgame ($\max(5, \lfloor |V| \cdot 0.08 \rfloor)$ conflicte).** Acesta defineste momentul in care algoritmul trece accentul de pe explorare pe intensificare puternica. In apropierea fezabilitatii, repararea locala devine mai importanta decat diversificarea.
- **Bugetul IGLS ($T_{IGLS} = 40$, extensibil pana la 200).** Repararea locala de baza ramane ieftina, insa in endgame merita extinsa pentru a creste sansele de eliminare completa a conflictelor.
- **Tabu tenure ($\tau = 7$).** O valoare prea mica permite cicluri scurte, iar una prea mare poate bloca inutil miscari bune. Valoarea 7 a oferit un echilibru bun intre libertatea cautarii si evitarea repetitiilor.
- **Politica de activare a repararii.** In endgame, repararea se aplica sistematic. In afara acestei zone, ea este activata probabilistic sau doar pentru descendenti suficient

de buni. Astfel, timpul de calcul este investit acolo unde are cel mai mare efect.

- **Limita de stagnare pentru restart (150 generatii).** Daca cea mai buna solutie nu se mai imbunatateste o perioada lunga, populatia este considerata convergenta. Restartul impiedica risipirea timpului intr-o regiune sterila a spatiului de cautare.
- **Compozitia restartului (5% elite, 70% aleator, 25% DSATUR).** Aceasta schema pastreaza progresul, dar reintroduce si structura buna, si diversitate. Este un echilibru intre memorie si resetare.
- **Injectie periodica de diversitate (la fiecare 100 generatii, 5% din populatie).** Aceasta interventie usoara previne uniformizarea tacita a populatiei si reduce probabilitatea unui restart complet prea frecvent.

Mesajul metodologic important este clar: performanta metodei nu depinde doar de ideea de algoritm genetic memetic, ci si de reglajul atent al operatorilor si al parametrilor care guverneaza interactiunea dintre ei.

4.4 Rezultatele metodei 1

Tabela 2: Rezultatele algoritmului genetic hibrid

Instanta	k_{ref}	k_{fez}	Extra	SR@ k_{ref} (%)	Conflicte medii la k_{ref}
queen7_7	7	7	0	100	0.0
myciel3	4	4	0	100	0.0
huck	11	11	0	100	0.0
myciel4	5	5	0	100	0.0
flat300_28_0	28	28	0	100	0.0
le450_5a	5	5	0	100	0.0
fpsol2.i.1	65	65	0	100	0.0
queen11_11	11	11	0	100	0.0
inithx.i.1	54	54	0	100	0.0
DSJC500.5	47	47	0	100	0.0
DSJC125.9	44	44	0	60	22.8
DSJC500.9	126	127	1	60	24.4
DSJC1000.9	223	224	1	20	44.8

Rezultatele evidentiaza foarte clar robustetea metodei genetice pe instantele usoare si medii. In aceste cazuri, algoritmul atinge sistematic valoarea de referinta, ceea ce arata ca mecanismele sale de explorare si intensificare functioneaza stabil.

Mult mai relevant este insa comportamentul pe instantele dificile. Chiar si atunci cand nu obtine fezabilitate la k_{ref} in toate rulajele, algoritmul ramane foarte aproape de aceasta.

Pentru DSJC500.9 si DSJC1000.9, o singura culoare suplimentara este suficienta pentru a recupera fezabilitatea.

Aceasta observatie sustine doua concluzii importante:

- metoda nu esueaza in mod haotic, ci converge in mod repetat in vecinatatea unor solutii aproape fezabile;
- combinatia dintre crossover structural, armonizare, mutatie adaptiva si IGLS tabu este eficienta pe instante dificile.

Valorile medii relativ mici ale conflictelor la k_{ref} confirma faptul ca algoritmul investeste bine bugetul de evaluari si valorifica eficient cautarea locala in zonele promitatoare.

5 Metoda 2: Algoritm Hibrid bazat pe Ant Colony Optimization

5.1 Descrierea metodei

A doua metoda foloseste o paradigma diferita fata de cea genetica. In loc sa opereze direct pe colorari complete, colonia de furnici construiesc ordonari de varfuri. Fiecare ordonare este apoi transformata intr-o colorare printr-o procedura greedy randomizata, folosind un numar fix de culori k .

Aceasta alegere este importanta din punct de vedere conceptual. Spatiul colorarilor complete este foarte mare si foarte simetric, in timp ce spatiul ordonarilor ofera un control mai clar asupra procesului constructiv. Daca o ordonare plaseaza devreme varfurile dificile sau foarte conectate, procedura greedy poate utiliza mai eficient culorile disponibile.

Feromonii sunt asociati perechilor *varf-pozitie* in ordonare. Astfel, algoritmul invata treptat ce varfuri merita sa fie plasate mai devreme in secventa de procesare. La fiecare iteratie, furnicile construiesc ordonari probabilistic, tinand cont atat de feromoni, cat si de informatie euristica, precum gradul varfurilor sau alte caracteristici locale.

Dupa constructia ordonarii, aceasta este convertita intr-o colorare printr-o strategie greedy randomizata. Solutia obtinuta este apoi imbunatatita printr-o procedura *Min-Conflicts* incrementală, care reduce rapid numarul de muchii conflictuale prin recolorari locale ieftine.

Atunci cand solutia ajunge aproape fezabila, metoda activeaza selectiv o cautare tabu de tip *TabuCol*. Aceasta este rezervata pentru momentele in care numarul de conflicte este deja mic, tocmai pentru a nu consuma inutil resurse in regiuni prea indepartate de fezabilitate.

Actualizarea feromonilor favorizeaza ordonarile care au condus la solutii bune, iar evaporarea previne rigidizarea cautarii. In caz de stagnare, algoritmul aplica reinitializari parțiale ale feromonilor si perturbări ghidate ale celei mai bune ordonari.

Prin urmare, metoda 2 este mai constructiva si, de regula, mai usoara per iteratie decat metoda genetica. Ea este foarte eficienta pe instante usoare si medii, dar poate deveni mai sensibila pe grafuri foarte dense, unde relatiile structurale complexe sunt mai greu de exploatat doar prin reprezentarea sub forma de ordonare.

5.2 Pseudocod

Algorithm 2 Algoritm ACO hibrid pentru colorarea fixed- k

Require: Graful $G = (V, E)$, numarul de culori k , buget de evaluari B

Ensure: Cea mai buna colorare gasita pentru k

```

1: Initializeaza urmele de feromoni
2:  $best \leftarrow \emptyset$ ,  $bestConf \leftarrow \infty$ 
3: while bugetul de evaluari nu este epuizat do
4:   for fiecare furnica do
5:     Construieste o ordonare de varfuri folosind feromoni si euristici
6:     Genereaza o colorare greedy randomizata cu  $k$  culori
7:     Aplica reparare incrementala Min-Conflicts
8:     Evalueaza numarul de conflicte
9:     if solutia curenta imbunatateste  $bestConf$  then
10:      Actualizeaza  $best$  si  $bestConf$ 
11:    end if
12:  end for
13:  if  $bestConf$  este sub pragul de intensificare then
14:    Aplica local search tabu selectiv asupra celei mai bune solutii
15:  end if
16:  Evapora si actualizeaza feromonii pe baza celei mai bune solutii din iteratie
17:  if se detecteaza stagnare then
18:    Aplica perturbare si restart partial al feromonilor
19:  end if
20: end while
21: return  $best$ 

```

5.3 Parametrii folositi si de ce conteaza

Si in cazul metodei ACO, selectia parametrilor este esentiala deoarece influenteaza direct raportul dintre explorare, exploatare si costul total al cautarii.

Tabela 3: Parametrii metodei ACO hibride

Parametru	Valoare	Rol
$ants_per_iter$	24	numarul de furnici pe iteratie
α	1.0	influenta feromonilor
β	3.0	influenta informatiei euristice
ρ	0.10	rata de evaporare a feromonilor
q_0	0.88	probabilitate de exploatare
τ_{min}	10^{-4}	limita inferioara pentru feromoni
τ_{max}	10.0	limita superioara pentru feromoni
τ_{prefix_len}	260	lungime de prefix folosita la update
$cand_size$	80	marimea listei restrictive de candidati
$cand_topdeg$	18	varfuri cu grad mare prioritizate
$greedy_rcl$	3	lista restransa de culori candidate
mc_steps	25000	pasi maximi pentru Min-Conflicts
mc_time	0.22 s	limita de timp pentru Min-Conflicts
$tabu_threshold$	30	prag de conflicte pentru activarea TabuCol
$tabu_every_iters$	12	frecventa activarii TabuCol
$tabu_steps$	60000	pasi maximi pentru TabuCol
$tabu_time$	0.30 s	limita de timp pentru TabuCol
$ilsp_perturb_frac$	0.10	fractia de varfuri perturbate la restart
$ilsp_perturb_bias$	0.80	bias spre zone conflictuale
$stall_iters$	90	prag pentru detectarea stagnarii
$mode$	prefix	modul euristic de constructie
B	400000	buget de evaluari per rulare
R	5	numar de rulari independente

Rolul practic al parametrilor poate fi sintetizat astfel:

- α si β . Acesti parametri controleaza raportul dintre experienta acumulata si ghidajul euristic. Daca α este prea mare, colonia se poate bloca prea devreme pe urme de feromoni insuficient validate. Daca β este prea mare, algoritmul se apropie de un comportament aproape determinist. Valorile alese favorizeaza euristica, dar lasa suficient loc invatarii prin feromoni.
- $\rho = 0.10$. O evaporare moderata pastreaza memorie utila pe termen mediu, fara a rigidiza excesiv colonia. Este un echilibru intre stabilitate si adaptare.
- $q_0 = 0.88$. Aceasta valoare incurajeaza exploatarea ordonarilor bune deja descoperite, dar nu elimina complet explorarea. Pentru problema de fata, aceasta balanta este

importanta, deoarece ordonarile foarte bune sunt rare.

- **Lista restrictiva de candidati** ($cand_size = 80$ si $cand_topdeg = 18$). Acest artificiu reduce costul constructiei si creste probabilitatea ca varfurile structurale importante, in special cele cu grad mare, sa fie tratate devreme.
- $greedy_rcl = 3$. Faza greedy nu este complet determinista. O lista restransa de culori candidate introduce diversitate, fara a compromite preferinta pentru alegeri locale bune.
- mc_steps si mc_time . *Min-Conflicts* este principalul operator ieftin de imbunatatire. Daca ii este alocat prea putin timp, solutiile raman prea conflictuale. Daca ii este alocat prea mult, colonia pierde din capacitatea de explorare globala.
- $tabu_threshold$, $tabu_every_iters$, $tabu_steps$, $tabu_time$. *TabuCol* este activat doar aproape de fezabilitate, deoarece este mai costisitor. Acest aspect arata din nou ca parametrizarea nu este un detaliu secundar, ci o parte esentiala a metodei.
- **Parametrii de perturbare si stagnare**. Acestia controleaza modul in care colonia iese din blocaj. O perturbare prea mica nu schimba suficient cautarea, iar una prea mare distruge informatia acumulata. Valorile alese mentin un compromis rezonabil.

Ca si in cazul metodei genetice, concluzia este aceeaasi: performanta nu vine doar din paradigma ACO in sine, ci din reglajul atent al procesului constructiv, al repararii locale si al mecanismelor de diversificare.

5.4 Rezultatele metodei 2

Tabela 4: Rezultatele algoritmului ACO hibrid

Instanta	k_{ref}	k_{fez}	Extra	SR@ k_{ref} (%)	Conflicte medii la k_{ref}
queen7_7	7	7	0	100	0.0
myciel3	4	4	0	100	0.0
huck	11	11	0	100	0.0
myciel4	5	5	0	100	0.0
flat300_28_0	28	28	0	100	0.0
le450_5a	5	5	0	100	0.0
fpsol2.i.1	65	65	0	100	0.0
queen11_11	11	11	0	100	0.0
inithx.i.1	54	54	0	100	0.0
DSJC500.5	47	47	0	100	0.0
DSJC125.9	44	45	1	20	31.2
DSJC500.9	126	127	1	20	80.2
DSJC1000.9	223	225	2	0	110.6

Rezultatele arata ca metoda ACO este foarte buna pe instantele usoare si medii, unde atinge constant valoarea de referinta. Chiar si pe `inithx.i.1` si `DSJC500.5`, comportamentul ramane foarte bun, ceea ce confirma ca abordarea prin ordonari si reparare locala poate fi competitiva si pe grafuri semnificativ mai dificile.

Diferenta fata de metoda genetica devine mai vizibila pe grafurile foarte dense. In special pentru `DSJC125.9`, `DSJC500.9` si mai ales `DSJC1000.9`, robustetea scade mai clar. Solutiile raman frecvent aproape fezabile, insa necesita mai des una sau chiar doua culori suplimentare pentru a obtine fezabilitatea completa.

Acest comportament este in acord cu designul algoritmului. Metoda lucreaza indirect, prin ordonari, iar pe grafuri extrem de dense interactiunile structurale dintre regiuni indepartate ale grafului devin mai greu de exploatat exclusiv prin aceasta reprezentare.

6 Metoda 3: Cautare Locala Hibrida cu Breakout Tabu

6.1 Descrierea metodei

A treia metoda propusa urmeaza o paradigma diferita fata de primele doua. Daca metoda genetica foloseste o populatie si operatori de recombinare, iar metoda ACO construiesc solutii printr-un mecanism probabilistic ghidat de feromoni, aceasta abordare este o metoda de tip *trajectory-based local search*. Cautarea porneste de la o singura colorare completa si o imbunatateste prin miscari locale succesive, adaptand in timp strategia de evitare a blocajelor.

Metoda este inspirata de ideile clasice din *TabuCol* si din cautarea locala cu mecanisme de penalizare, fiind proiectata pentru varianta fixed- k a problemei. Reprezentarea solutiei este aceeaasi ca in functia obiectiv generala: fiecare varf primeste permanent o culoare din multimea $\{0, 1, \dots, k - 1\}$. Spre deosebire de *PartialCol*, unde unele varfuri pot ramane temporar necolorate, aici solutia este intotdeauna completa, iar calitatea ei este data de numarul de muchii conflictuale.

Punctul de plecare este o colorare construita prin *DSATUR*, redusa apoi la valoarea curenta a lui k printr-o procedura greedy de minimizare a conflictelor locale. Aceasta initializare ofera o solutie de start rezonabila, dar nu garanteaza fezabilitatea. Din acest punct, algoritmul aplica o cautare tabu intensiva: la fiecare pas sunt selectate varfuri aflate in conflict, iar pentru fiecare dintre acestea sunt evaluate recolorari candidate. Miscarea aleasa este cea care reduce cel mai mult numarul de conflicte, cu respectarea restrictiilor tabu.

Pentru eficienta, metoda foloseste o matrice incrementală

$$A[v][c],$$

unde $A[v][c]$ reprezinta numarul de vecini ai varfului v care au culoarea c . Astfel, efectul brut al recolorarii lui v din culoarea curenta in culoarea c poate fi calculat rapid, fara reevaluarea intregii solutii. Aceasta structura este esentiala pentru grafurile dense, deoarece permite efectuarea unui numar mare de miscari locale in timpul disponibil.

Componenta distinctiva a metodei este mecanismul de tip *breakout*. Atunci cand cautarea stagneaza, algoritmul maresta penalizarile asociate muchiilor care sunt conflictuale in solutia curenta. In consecinta, functia de evaluare nu mai tine cont doar de numarul brut de conflicte, ci si de greutatea acestora:

$$F(C) = \sum_{(u,v) \in E} p_{uv} \cdot \mathbb{I}[C(u) = C(v)],$$

unde p_{uv} este penalizarea asociata muchiei (u, v) . Initial, toate penalizarile sunt egale cu 1. In timpul cautarii, muchiile care apar repetat in conflicte devin mai costisitoare, fortand algoritmul sa paraseasca optime locale recurente.

Tabu tenure-ul este adaptiv. In perioadele de progres, durata tabu este redusa pentru a permite rafinarea fina a solutiei. In perioadele de stagnare, aceasta este crescuta, ceea ce incurajeaza miscari mai exploratorii si reduce riscul ciclurilor scurte. Daca stagnarea persista, metoda aplica restarturi strategice, dar nu complet aleatoare. Sunt memorate cateva solutii de elite, iar restartul porneste de regula de la o astfel de solutie perturbata controlat.

Prin urmare, metoda 3 combina patru mecanisme complementare:

- cautare tabu pe colorari complete;
- evaluare incrementală a recolorarilor;
- penalizari de tip breakout pentru iesirea din optime locale;
- memorie de elite si restarturi strategice.

Aceasta combinatie o face semnificativ diferita atat de metoda genetica, cat si de ACO. Ea nu foloseste populatie, crossover sau feromoni, ci se bazeaza pe intensificare locala puternica si mecanisme adaptive de diversificare.

6.2 Pseudocod

Algorithm 3 Cautare locala hibrida cu Breakout Tabu pentru colorarea fixed- k

Require: Graful $G = (V, E)$, numarul de culori k , limita de timp sau bugetul de evaluari B

Ensure: Cea mai buna colorare gasita

```

1: Construieste o solutie initiala folosind DSATUR si reducere greedy la  $k$  culori
2: Initializeaza matricea incrementala  $A[v][c]$ 
3: Initializeaza penalizarile muchiilor  $p_{uv} \leftarrow 1$ 
4: Memoreaza solutia curenta ca cea mai buna solutie initiala
5: while conditiile de oprire nu sunt indeplinite si solutia nu este fezabila do
6:   Selecteaza varfuri candidate dintre varfurile aflate in conflict
7:   for fiecare varf candidat  $v$  do
8:     for fiecare culoare candidata  $c \neq C(v)$  do
9:       Calculeaza variatia bruta a conflictelor folosind  $A[v][c]$ 
10:      Calculeaza variatia ponderata folosind penalizarile muchiilor
11:      Ignora miscarea daca este tabu si nu satisface criteriul de aspiratie
12:    end for
13:  end for
14:  Aplica cea mai buna miscare admisibila
15:  Actualizeaza incremental matricea  $A[v][c]$  si numarul de conflicte
16:  Actualizeaza lista tabu si durata tabu adaptiva
17:  if solutia curenta imbunatateste cea mai buna solutie then
18:    Actualizeaza cea mai buna solutie si memoria de elite
19:  end if
20:  if se detecteaza stagnare then
21:    Mareste penalizarile muchiilor aflate in conflict
22:    Creste tabu tenure-ul pentru diversificare
23:  end if
24:  if stagnarea persista then
25:    Executa restart strategic dintr-o solutie de elita perturbata
26:  end if
27: end while
28: return cea mai buna colorare gasita

```

6.3 Parametrii folositi si de ce conteaza

Metoda AHGC cu Breakout Tabu este controlata de parametri care regleaza direct intensitatea cautarii locale si capacitatea de iesire din stagnare.

- **Numarul de varfuri candidate.** La fiecare iteratie nu sunt evaluate toate varfurile, ci doar un esantion dintre varfurile aflate in conflict, prioritizand varfurile cu contributie conflictuala ridicata. Aceasta reduce costul unei iteratii si directioneaza

cautarea spre zonele relevante.

- **Tabu tenure de baza.** Acesta stabileste durata minima pentru care o miscare inversa este interzisa. O valoare prea mica favorizeaza ciclurile, iar una prea mare poate bloca miscari utile. De aceea, metoda porneste de la o valoare moderata si o adapteaza dinamic.
- **Tabu tenure maxim.** Limita superioara previne cresterea necontrolata a restrictiilor tabu in perioade lungi de stagnare. Astfel, diversificarea este posibila fara ca spatiul de miscari admisibile sa devina prea restrans.
- **Pragul de stagnare pentru breakout.** Daca cea mai buna solutie nu se imbunatateste un numar suficient de mare de iteratii, se considera ca algoritmul a intrat intr-o regiune locala dificila. In acel moment sunt penalizate muchiile conflictuale curente.
- **Pragul de restart strategic.** Daca penalizarile si cresterea tabu tenure-ului nu sunt suficiente, algoritmul reporneste cautarea dintr-o solutie de elita perturbata. Acest mecanism combina memoria cautarii cu diversificarea.
- **Dimensiunea memoriei de elite.** Sunt pastrate cateva dintre cele mai bune solutii gasite. O memorie prea mica reduce diversitatea restarturilor, iar una prea mare poate pastra solutii mediocre. O dimensiune redusa este suficienta pentru a mentine directii promitatoare.
- **Penalizarile muchiilor.** Acestea sunt esentiale pentru mecanismul breakout. Muchiile care apar repetat in conflicte sunt facute mai costisitoare, ceea ce modifica local peisajul de cautare si impiedica revenirea in aceleasi optime locale.

Prin acesti parametri, metoda reuseste sa imbine intensificarea agresiva cu diversificarea adaptiva. In loc sa aplice restarturi complet aleatoare, algoritmul modifica treptat presiunea de cautare si foloseste informatia acumulata despre conflictele recurente.

6.4 Rezultatele metodei 3

Tabela 5: Rezultatele metodei AHGC cu Breakout Tabu

Instanta	k_{ref}	k_{fez}	Extra	SR@ k_{ref} (%)	Conflicte medii la k_{ref}
queen7_7	7	7	0	100	0.0
myciel3	4	4	0	100	0.0
huck	11	11	0	100	0.0
myciel4	5	5	0	100	0.0
flat300_28_0	28	28	0	100	0.0
le450_5a	5	5	0	100	0.0
fpsol2.i.1	65	65	0	100	0.0
queen11_11	11	11	0	100	0.0
inithx.i.1	54	54	0	100	0.0
DSJC500.5	47	47	0	100	0.0
DSJC125.9	44	44	0	100	0.0
DSJC500.9	126	127	1	60	24.0
DSJC1000.9	223	224	1	40	42.0

Rezultatele metodei 3 sunt apropiate de cele obtinute de algoritmul genetic hibrid, ceea ce confirma ca o cautare locala adaptiva bine proiectata poate concura cu metode populationale complexe. Pe instantele usoare si medii, metoda atinge constant valoarea de referinta si obtine rate de succes de 100%.

Pe instantele dificile, comportamentul ramane stabil. In particular, metoda reuseste sa pastreze fezabilitatea la valoarea de referinta pentru majoritatea grafurilor si are nevoie de o singura culoare suplimentara pe instantele cele mai dificile. Acest rezultat este important deoarece arata ca mecanismele de breakout si restarturile strategice reduc riscul stagnarii in optime locale profunde.

Comparativ cu metoda ACO, AHGC este mai puternica pe grafuri dense deoarece actioneaza direct asupra conflictelor din colorarea completa. Comparativ cu metoda genetica, nu beneficiaza de recombinare intre solutii diferite, dar compenseaza printr-o intensificare locala mai directa si prin penalizarea adaptiva a conflictelor recurente.

7 Comparatie intre cele trei metode

Cele trei metode rezolva aceeasi problema, dar o ataca din perspective metaeuristice diferite. Algoritmul genetic hibrid lucreaza direct in spatiul colorarilor complete si combina recombinarea structurala cu repararea locala agresiva. Algoritmul ACO hibrid lucreaza indirect, prin ordonari de varfuri, si se bazeaza pe constructie probabilistica, colorare greedy si optimizare locala selectiva. Metoda AHGC cu Breakout Tabu foloseste o singura

traectorie de cautare, dar o controleaza adaptiv prin tabu tenure, penalizari ale conflictelor recurente si restarturi din solutii de elita.

Tabela 6: Comparatie sintetica intre cele trei metode

Aspect	GA hibrid	ACO hibrid	AHGC Breakout Tabu
Spatiu de cautare	colorari complete	ordonari de varfuri	colorari complete
Tip de cautare	populationala	constructiva, swarm	traectorie locala
Explorare globala	crossover, mutatie, elite	feromoni si randomizare	restarturi si breakout
Intensificare	IGLS tabu	Min-Conflicts si TabuCol	tabu search ponderat
Memorie	populatie si elite	urme de feromoni	elite si penalizari
Evadare din optime locale	puternica	moderata	puternica
Robustete pe grafuri dificile	ridicata	medie	ridicata
Sensibilitate la densitate	mica	mai mare	mica-medie
Cost pe iteratie	ridicat	redus	mediu
Instante avantajate	medii, dificile, dense	usoare, medii, structurate	dificile, dense, aproape fezabile

Din rezultatele experimentale se observa ca metoda genetica si metoda AHGC au comportamente apropiate pe instantele dificile. Ambele lucreaza direct asupra colorarilor complete si pot exploata eficient informatia locala despre conflicte. Diferenta principala este mecanismul de diversificare: GA-ul foloseste recombinare si populatie, in timp ce AHGC modifica peisajul de cautare prin penalizari de tip breakout si restarturi strategice.

Metoda ACO are un avantaj conceptual si computational pe instantele usoare si medii, unde ordonarile bune sunt suficiente pentru a ghida eficient colorarea greedy. Pe grafuri foarte dense, insa, performanta sa scade mai vizibil, deoarece interactiunile dintre varfuri sunt mai greu de captat doar prin ordonare si feromoni.

Comparatia globala arata ca cele trei metode sunt complementare:

- **GA hibrid** este cea mai echilibrata metoda, cu rezultate robuste si mecanisme variate de explorare si intensificare;
- **ACO hibrid** este eficient pe instante structurate si are un cost redus pe iteratie;
- **AHGC Breakout Tabu** este o metoda trajectory-based puternica, foarte potrivita pentru rafinarea agresiva a colorarilor complete si pentru iesirea din optime locale recurente.

Din punct de vedere al calitatii solutiilor, AHGC se apropie de performanta metodei genetice si imbunatateste robustetea fata de ACO pe instantele dense. In acelasi timp, faptul ca nu foloseste populatie sau crossover o face suficient de diferita metodologic pentru a reprezenta o a treia directie relevanta de investigatie.

8 Concluzie

Lucrarea a prezentat trei metode hibride pentru problema colorarii grafurilor in varianta *fixed-k*, toate proiectate pentru a functiona sub un buget strict de evaluari. Prima metoda,

bazata pe un algoritm genetic memetic, s-a dovedit foarte robusta, in special pe instante dificile si dense. A doua metoda, bazata pe *Ant Colony Optimization*, s-a remarcat prin eficienta buna si printr-un comportament solid pe instante usoare si medii. A treia metoda, AHGC cu Breakout Tabu, a demonstrat ca o cautare locala adaptiva, fara populatie si fara feromoni, poate obtine rezultate comparabile cu metoda genetica atunci cand este sustinuta de evaluare incrementala, penalizari dinamice si restarturi strategice.

Contributia principala a lucrarii nu consta doar in implementarea unor metode functionale, ci si in compararea a trei paradigme metaeuristice distincte: cautare evolutionara, inteligenta de roi si cautare locala adaptiva. Fiecare abordare a fost analizata prin prisma mecanismelor interne, a pseudocodului, a parametrilor si a rezultatelor obtinute. Acest lucru este important deoarece, pentru probleme combinatoriale dificile, performanta finala depinde atat de paradigma aleasa, cat si de modul concret in care aceasta este hibridizata si reglata.

Rezultatele obtinute arata ca toate cele trei metode pot atinge performante foarte bune pe instante usoare si medii. Diferentele apar mai clar pe grafurile dense si foarte dificile. Algoritmul genetic si AHGC sunt mai robuste in apropierea pragului cromatic, in timp ce ACO ramane competitiv, dar mai sensibil la densitate. In cazurile in care fezabilitatea la k_{ref} nu este atinsa constant, metodele robuste recupereaza fezabilitatea printr-un numar foarte mic de culori suplimentare, in general una sau doua.

Ca directii viitoare, merita investigata o combinatie mai stransa a celor trei paradigme. De exemplu, ordonarile produse de ACO ar putea fi utilizate pentru initializarea populatiei genetice, iar solutiile de elita obtinute de GA ar putea servi ca puncte de restart pentru AHGC. De asemenea, mecanisme de penalizare breakout pot fi integrate in operatorii de reparare ai metodelor populationale. O alta directie utila este adaptarea automata a parametrilor pe baza indicatorilor de progres observati in timpul rularii.

In ansamblu, rezultatele confirma ca metaeuristicile hibride raman o abordare foarte eficienta pentru problema colorarii grafurilor, mai ales atunci cand combina reprezentari potrivite, evaluare incrementala si mecanisme explicite de evitare a stagnarii.

9 Bibliografie

Bibliografie

- [1] M. R. Garey and D. S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.
- [2] D. Brélaz. A new method for colouring the vertices of a graph. *Communications of the ACM*, 22(4):251–256, 1979.
- [3] A. Hertz and D. de Werra. Tabu search for graph coloring. *Computing*, 39(4):345–351, 1987.
- [4] P. Galinier and J.-K. Hao. Hybrid evolutionary algorithms for graph coloring. *Journal of Combinatorial Optimization*, 3(4):379–397, 1999.
- [5] M. Dorigo and T. Stützle. *Ant Colony Optimization*. MIT Press, 2004.
- [6] I. Blöchliger and N. Zufferey. A graph coloring heuristic using partial solutions and a reactive tabu scheme. *Computers & Operations Research*, 35(3):960–975, 2008.
- [7] M. Chiarandini, I. Dumitrescu, and T. Stützle. Stochastic local search algorithms for the graph colouring problem. In T. F. Gonzalez (Ed.), *Handbook of Approximation Algorithms and Metaheuristics*. Chapman & Hall/CRC, 2007.
- [8] F. Glover. Tabu Search – Part I. *ORSA Journal on Computing*, 1(3):190–206, 1989.
- [9] E. Malaguti and P. Toth. A survey of graph coloring problems. *EURO Journal on Computational Optimization*, 1(1–2):1–46, 2010.
- [10] D. S. Johnson and M. A. Trick (Eds.). *DIMACS Implementation Challenges: Coloring and Satisfiability*. American Mathematical Society, 1996.